Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

Lectures

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forecast, boosting
- Lecture 14: Unsupervised learning and clustering algorithms
- Lecture 15: Unsupervised learning and PCA
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

11. Naive Bayes, LDA, QDA

2102575

Suwichaya Suwanwimolkul[1]

[1]Electrical Engineering
Chulalongkorn University

Semester II, 2025

1 Table of Contents

2 Bayesian decision theory

3 Naive Bayes

4 Decision boundary

5 Discriminant function

6 Types of naive Bayes

7 Extend to multiple observations for training

8 Training
    Training of LDA

9 Quadratic discriminant analysis (QDA)

Generative Classifiers

1 Table of Contents

2 Bayesian decision theory

3 Naive Bayes

4 Decision boundary

5 Discriminant function

6 Types of naive Bayes

7 Extend to multiple observations for training

8 Training

9 Quadratic discriminant analysis (QDA)

Terminology in Bayesian theory

- Prior probability $p(x)$ is the knowledge we have *before* looking at an observed x.
- Class likelihood or class-conditional density $p(x|y)$ gives distribution information of features in each class (once we know the response variable belong to the class y)
- Evidence $p(x)$ is the marginal probability that a value x is observed (regardless of the class of Y) —can be computed using total probability.
- Posterior probability $p(y|x)$ is the likelihood of response after x is seen.

Bayes' rule: $$\text{Posterior} = \frac{\text{class likelihood} \times \text{prior}}{\text{evidence}}$$

$$p(y = c|x) = \frac{p(x|y = c)p(y = c)}{p(x)}$$

From a discriminative task to generative classifiers (naive Bayes)

A common goal in discriminative task is

$$\hat{c} = \underset{c \in \{1,2,\ldots,C\}}{\textbf{argmax}} \; p(x|y=c)$$

which is to find the class c that has the highest likelihood...

Let's turn this problem into a **generative task**:

$$\hat{c} = \underset{c \in \{1,2,\ldots,C\}}{\textbf{argmax}} \; p(y=c|x) = \frac{p(x|y=c)p(y=c)}{p(x)}.$$

To solve for c, one needs to calculate the following terms:

- Prior: $p(y=c)$ can be computed using the fraction of each class from the training data.
- Likelihood $p(x|y=c)$ can be computed using the distribution of input features x, which can be partially parameterized by the condition of each class $y=c$ for $c \in \{1, 2, ..., C\}$.
- This setting we are interested in identifying the class of a given input x.

Decision boundary

A discriminant function is used to find the decision boundary. The discriminant function can be defined as $p(y = c|x)$ or a function that is proportional to $p(y = c|x)$

$$g_c(x) = p(y = c|x) \quad \text{or} \quad g_c(x) \propto p(y = c|x)$$

Then, the optimization will turn into

$$\hat{c} = \underset{c \in \{1,2,\ldots,C\}}{\textbf{argmax}} \; p(y = c|x) \propto p(x|y = c)p(y = c) \iff \hat{c} = \underset{c \in \{1,2,\ldots,C\}}{\textbf{argmax}} \; g_c(x).$$

The goal is to find c that gives the highest discriminant function which represents the posterior distribution.

Decision rule: suppose that $c \in \{0, 1\}$.

- Classifying to $c = 0$

$$\begin{aligned} p(y = 0|x) > p(y = 1|x) &\iff \log p(x|y = 0) + \log p(y = 0) > \log p(x|y = 1) + \log p(y = 1) \\ &\iff \log g_{c=0}(x) > \log g_{c=1}(x) \end{aligned}$$

- Classifying to $c = 1$

$$\begin{aligned} p(y = 1|x) > p(y = 0|x) &\iff \log p(x|y = 1) + \log p(y = 1) > \log p(x|y = 0) + \log p(y = 0) \\ &\iff \log g_{c=1}(x) > \log g_{c=0}(x) \end{aligned}$$

Discriminant function

What define $g_c(x)$?

$$\begin{aligned} g_c(x) &\propto p(y=c|x) \\ &\propto p(x|y=c)p(y=c) \end{aligned}$$

1. Prior: $p(y=c)$ can be computed using the fraction of each class from the training data. It can be seen as the weight for each class.
2. Likelihood $p(x|y=c)$ can be computed in the following manners:

→ if we did not assume any distribution on $p(x|y=c)$, then the generative task is just a **Naive Bayes**.

→ if $p(x|y=c) := \begin{cases} q_c & \text{if } x=1 \\ 1-q_c & \text{otherwise} \end{cases}$
where x can only be either 0 or 1, then we call it **Bernoulli Naive Bayes**. Note that q_c is the corresponding event when $x=1$ and can be controlled by $y=c$.

→ if $p(x|y=c) := \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp - \frac{(x-\mu_c)^2}{2\sigma_c^2}$, then we call it **Gaussian Naive Bayes**. Notice that μ_c, σ_c^2 are the parameters associated with the distribution of input features x and can be controlled by $y=c$.

Example: Bernoulli Naive Bayes

Suppose that $y \sim \mathcal{U}(0, C)$ and $p(x|y = c)$ is a Bernoulli distribution parameterized according to the following:

$$p(x|y = c) := \begin{cases} q_c & \text{if } x = 1 \\ 1 - q_c & \text{otherwise} \end{cases}$$

where the value of q_c depends on the value of $c \in \{0, 1, ..., C\}$.
How to formulate the discriminant function $g_c(\cdot)$?

$$\begin{aligned} g_c(x) &\propto p(x|y = c)p(y = c) \\ &\propto (q_c)^x (1 - q_c)^{1-x} \frac{1}{C} \end{aligned}$$

To simplify the above formulation for computation, we can compute $g_c(x)$ in logarithmic scale:

$$\begin{aligned} g_c(x) &= \log (q_c)^x (1 - q_c)^{1-x} \frac{1}{C} \\ &= x \log (q_c) + (1 - x) \log (1 - q_c) - \log C \\ &= x \left(\log (q_c) - \log (1 - q_c)\right) + \log (1 - q_c) - \log C \end{aligned}$$

Note $g_c(c)$ is a linear function in x.

Example: Gaussian Naive Bayes

Suppose that $y \sim \mathcal{U}(0, C)$ and $p(x|y=c)$ is parameterized according to the following:

$$p(x|y=c) := \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp - \frac{(x-\mu_c)^2}{2\sigma_c^2}$$

where μ_c and σ_c depends on the value of $c \in \{0, 1, ..., C\}$. How to formulate $g_c(\cdot)$?

$$\begin{aligned} g_c(x) &\propto p(x|y=c)p(y=c) \\ &\propto \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp - \frac{(x-\mu_c)^2}{2\sigma_c^2} \frac{1}{C} \end{aligned}$$

To simplify the above formulation for computation, we can compute $g_c(x)$ in logarithmic scale:

$$\begin{aligned} g_c(x) &= \log\left(\frac{1}{\sqrt{2\pi\sigma_c^2}}\right) + \log\left(\exp - \frac{(x-\mu_c)^2}{2\sigma_c^2}\right) + \log\left(\frac{1}{C}\right) \\ &= -\log\sqrt{2\pi\sigma_c^2} - \frac{(x-\mu_c)^2}{2\sigma_c^2} - \log C \\ &= -\frac{1}{2\sigma_c^2}x^2 + \frac{\mu_c}{\sigma_c^2}x - \frac{\mu_c^2}{2\sigma_c^2} - \log\sqrt{2\pi\sigma_c^2} - \log C \end{aligned}$$

If $\sigma_c^2 = \sigma^2$ for all $c \in \{1, 2, ..., C\}$, then the term $\frac{1}{2\sigma^2}x^2$ can be ignored, and thus, $g_c(x)$ will be a linear function in x. Otherwise, $g_c(x)$ is quadratic function in x.

Types of naive Bayes

A common goal in discriminative classification is

$$\hat{c} = \underset{c=1,2,\ldots,C}{\textbf{argmax}} \; p(y=c|x) = \frac{p(x|y=c)p(y=c)}{p(x)} = g_c(x|\mu_c, \Sigma_c)$$

Inference. The discriminant function $g_l(x|\mu_c, \Sigma_c)$.

- Naive Bayes: $\log g_c(x|\mu_c, \Sigma_c)$ is directly derived from

$$g_c(x|\mu_c, \Sigma_c) \propto \prod_{j=1}^{m} p(x_j|y_j)p(y_j)$$

- LDA : $\log g_c(x|\mu_c, \Sigma_c)$ is a linear discriminant function, *i.e.*,

$$\log g_c(x|\mu_c, \Sigma_c) \approx \beta_1 x + \beta_0$$

- QDA : $\log g_c(x|\mu_c, \Sigma_c)$ is a quadratic discriminant function.

$$\log g_c(x|\mu_c, \Sigma_c) \approx \beta_2 x^2 + \beta_1 x + \beta_0$$

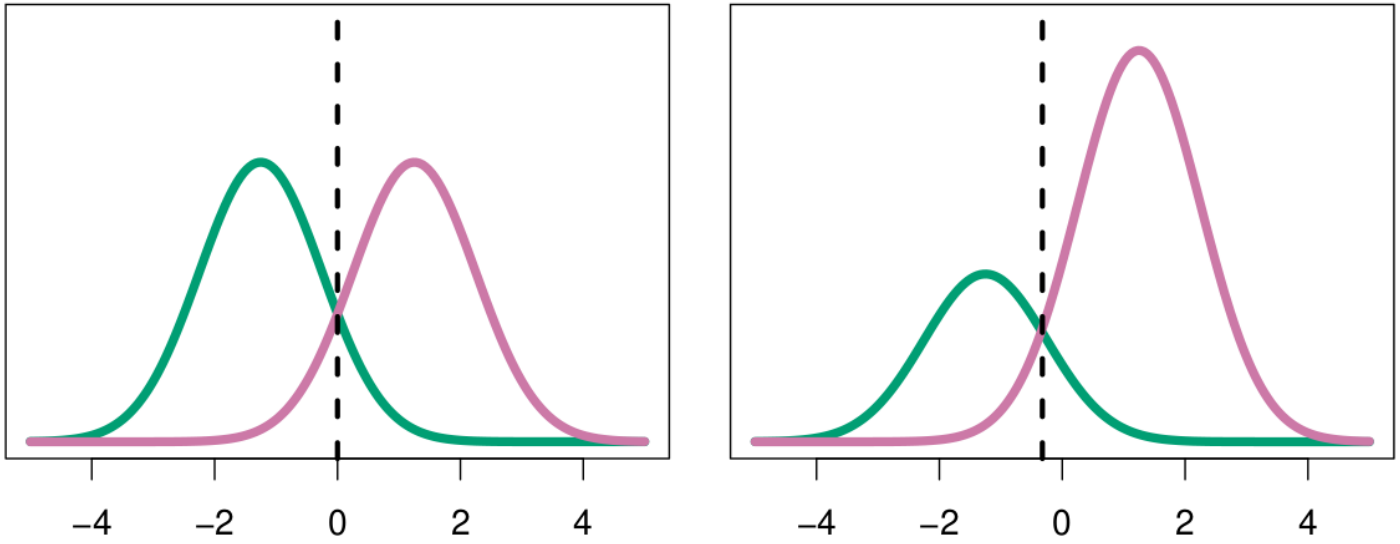
Side notes:

- The parameters associated with $x|y$, *e.g.*, q, μ, Σ, are estimated from sample mean and variance of the training data.
- The estimated mean and variance depends on how x is divided into classes.

Bayes decision boundary: scalar case

An example of posterior densities for two cases of prior probabilities;

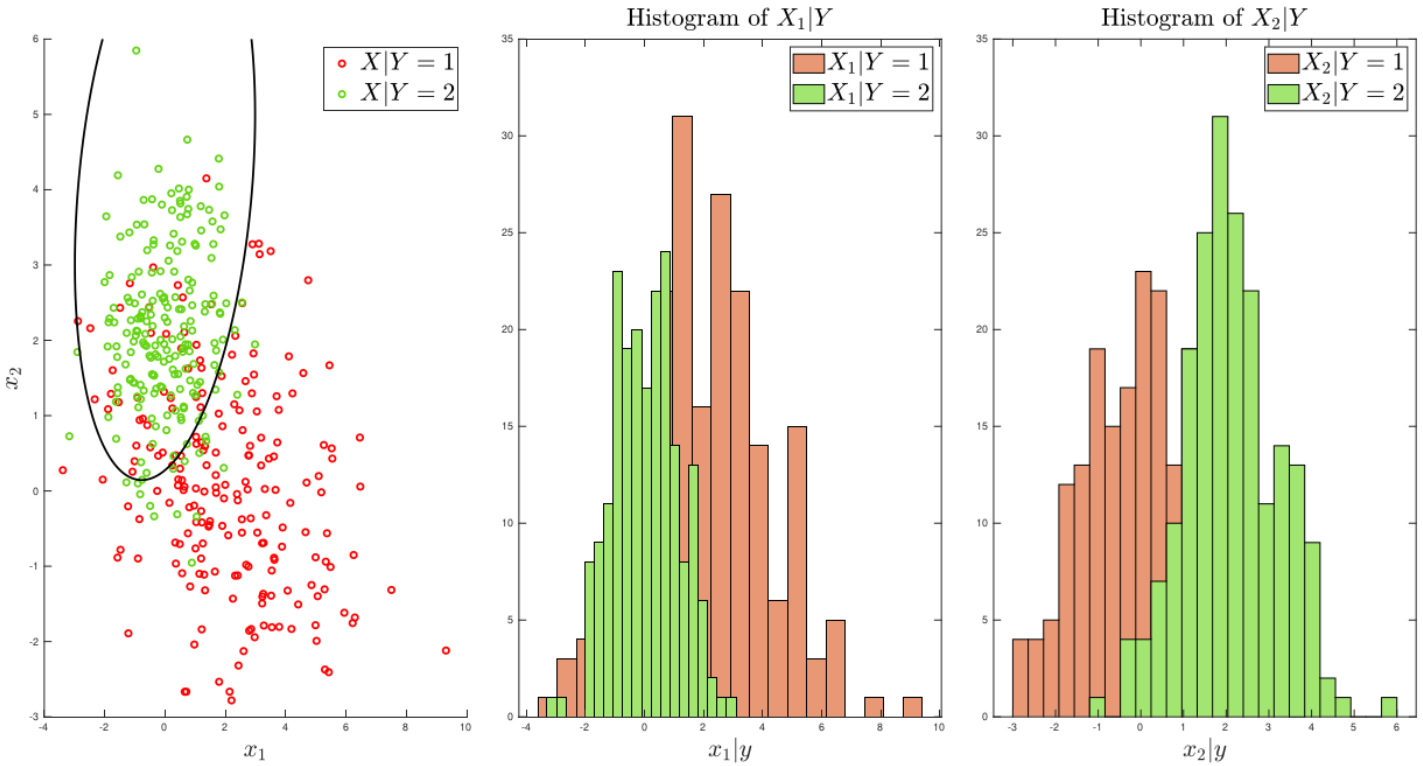
$P(Y = 1) = P(Y = 2) = 0.5$ $\quad$ $P(Y = 1) = 0.3, P(Y = 2) = 0.7$



- For a given test observation x, assign to the class for which the density is highest either class 1 or class 2.
- When the prior of class 2 is higher, the decision boundary is shifted to the left (more favor to class 2).

Example: conditional 2D Gaussians

$$X|y=1 \sim \mathcal{N}\left(\begin{bmatrix}2\\0\end{bmatrix}, \begin{bmatrix}4 & -1\\-1 & 2\end{bmatrix}\right) \text{ and } X|y=2 \sim \mathcal{N}\left(\begin{bmatrix}0\\2\end{bmatrix}, \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}\right)$$



- Bayes decision boundary of Gaussian case leads to a quadratic function.
- What happen to the decision boundary as $p(y=1)$ increases ?

Extend to multiple observations for training

- Suppose that our training dataset contains m observation pairs of input features and labels (x_j, y_j) for $j = 1, ..., m$.
- Then, the posterior distribution is

$$\prod_{j=1}^{m} p(y_j|x_j) \propto \prod_{j=1}^{m} p(x_j|y_j)p(y_j)$$

Therefore, the discriminant function for a training dataset can be formulated as follows:

$$g_c(x|\mu_c, \Sigma_c) \propto \prod_{j=1}^{m} p(x_j|y_j)p(y_j)$$

- **Multi-dimensional case.**

However, in most cases, x_j is an n-dimensional input features, *i.e.*, $x_j := [x_j^i]_{i\in[n]}$. Thus, we can write $p(x_j|y_j)$ as

$$p(x_j|y_j) = p([x_j^i]_{i\in[n]}|y_j)$$

Therefore, the discriminant function can be formulated as follows:

$$g_c(x|\mu_c, \Sigma_c) \propto \prod_{j=1}^{m} p([x_j^i]_{i\in[n]}|y_j)$$

Training example

- Training of LDA

Training of LDA: problem settings

Assumption.
LDA applies **a Gaussian assumption with equal covariance** on $p(x|y=c)$.

- $p(x|y=c)$ is a Gaussian with mean μ_c and equal covariance Σ for $c = 1, ..., C$
- Let $p_c(x)$ denotes the posterior probability $\triangleq P(y=c|x) \propto \frac{p(x|y=l)p(y=l)}{p(x)}$:

$$p_c(x) = \frac{\exp\left(-\frac{1}{2}(x-\mu_c)^T\Sigma^{-1}(x-\mu_c)\right)\pi_c}{\sum_{k=1}^{C}\exp\left(-\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k)\right)\pi_k}$$

- **Note:** the denominators are the same, regardless of the density and classes.

Inference.
Take the $\log p_c(x)$ and neglect the term $x^T\Sigma^{-1}x$, we predict the class c for which a discriminant function (defined below) is the highest among $c = 1, 2, ...C$

$$\begin{aligned}\text{Discriminant function: } g_c(x) &= x^T\Sigma^{-1}\mu_c - \frac{1}{2}\mu_c^T\Sigma^{-1}\mu_c + \log\pi_c \\ &= x^T\boldsymbol{\beta}_1 + \beta_0\end{aligned}$$

Decision boundary of LDA: Training

Training.

- The decision boundary between a and b happens when the two classes have equal discriminant values.

$$
\begin{aligned}
& g_a(x) = g_b(x) \iff g_a(x) - g_b(x) = 0 \\
& \iff \left(x^T \Sigma^{-1} \mu_a - \frac{1}{2} \mu_a^T \Sigma^{-1} \mu_a + \log \pi_a\right) - \left(x^T \Sigma^{-1} \mu_b - \frac{1}{2} \mu_b^T \Sigma^{-1} \mu_b + \log \pi_b\right) = 0 \\
& \iff x^T \hat{\Sigma}^{-1} (\hat{\mu}_a - \hat{\mu}_b) - \frac{1}{2} (\hat{\mu}_a^T \hat{\Sigma}^{-1} \hat{\mu}_a - \hat{\mu}_b^T \hat{\Sigma}^{-1} \hat{\mu}_b) + \log \frac{\hat{\pi}_a}{\hat{\pi}_b} = 0
\end{aligned}
$$

- The decision boundary is linear in x.
- When prior densities are equal, the boundary is the midpoint of any two sample means of x associated with each pair of classes: $x = \frac{1}{2}(\hat{\mu}_a + \hat{\mu}_b)$. This can be derived from

$$
\begin{aligned}
x^T \hat{\Sigma}^{-1} (\hat{\mu}_a - \hat{\mu}_b) - \frac{1}{2} (\hat{\mu}_a^T \hat{\Sigma}^{-1} \hat{\mu}_a - \hat{\mu}_b^T \hat{\Sigma}^{-1} \hat{\mu}_b) = 0 \\
x^T \hat{\Sigma}^{-1} (\hat{\mu}_a - \hat{\mu}_b) = \frac{1}{2} (\hat{\mu}_a + \hat{\mu}_b)^T \hat{\Sigma}^{-1} (\hat{\mu}_a - \hat{\mu}_b)
\end{aligned}
$$

- This mean that μ_c for x associated with each class can be calculated from

$$\hat{\mu}_c = \text{sample mean of } X \text{ in class } c$$

- Then, the variance $\hat{\Sigma} = \frac{1}{m-C} \sum_{c=1}^{C} \sum_{j \in [m] | y_j = c} (x_j - \hat{\mu}_c)(x_j - \hat{\mu}_c)^T$.

Training phase: compute $p(y=c)$, $\hat{\mu}_c$, and $\hat{\Sigma}$

1. **Partition data**:

   Partition y_j for $j \in [m]$ according to its associated labels y_j: $\mathcal{Y}_c = \{y_j, j \in [m] | y_j = c\}$.
   For example: $\mathcal{Y}_{c=0} = \{y_j, j \in [m] | y_j = 0\}$, $\mathcal{Y}_{c=1} = \{y_j, j \in [m] | y_j = 1\}$, ...,
   $\mathcal{Y}_{c=C} = \{y_j, j \in [m] | y_j = C\}$.

   Partition x_j for $j \in [m]$ according to its associated labels y_j: $\mathcal{X}_c = \{x_j, j \in [m] | y_j = c\}$.
   For example: $\mathcal{X}_{c=0} = \{x_j, j \in [m] | y_j = 0\}$, $\mathcal{X}_{c=1} = \{x_j, j \in [m] | y_j = 1\}$, ...,
   $\mathcal{X}_{c=C} = \{x_j, j \in [m] | y_j = C\}$.
2. **Compute** $p(y=c)$ for all $c \in \{1, 2, ..., C\}$.
   $p(y=0) = \frac{|\mathcal{Y}_{c=0}|}{m}$, $p(y=1) = \frac{|\mathcal{Y}_{c=1}|}{m}$, ..., $p(y=C) = \frac{|\mathcal{Y}_{c=C}|}{m}$
3. **Compute** $\hat{\mu}_c$ for all $c \in \{1, 2, ..., C\}$.
   $\hat{\mu}_{c=0} = \frac{\sum_{x \in \mathcal{X}_{c=0}} x}{|\mathcal{X}_{c=0}|}$, $\hat{\mu}_{c=1} = \frac{\sum_{x \in \mathcal{X}_{c=1}} x}{|\mathcal{X}_{c=1}|}$, ..., $\hat{\mu}_{c=C} = \frac{\sum_{x \in \mathcal{X}_{c=C}} x}{|\mathcal{X}_{c=C}|}$
4. **Compute** $\hat{\Sigma}$...

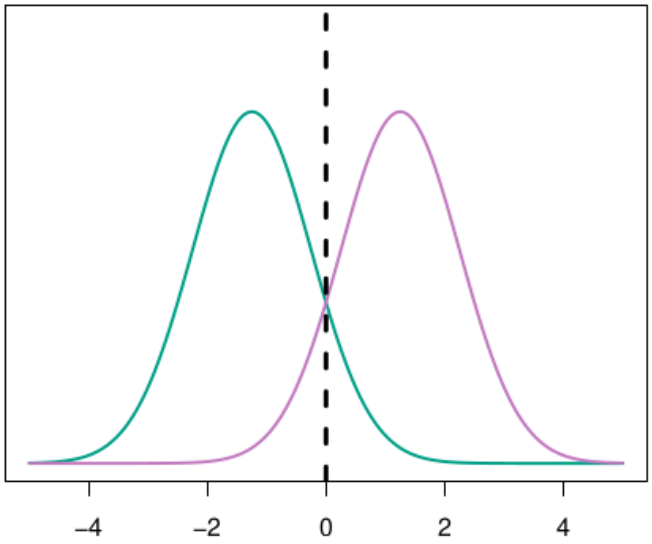
$$\hat{\Sigma} = \frac{1}{m-C} \sum_{c=1}^{C} \sum_{x \in \mathcal{X}_c} (x - \hat{\mu}_c)(x - \hat{\mu}_c)^T$$

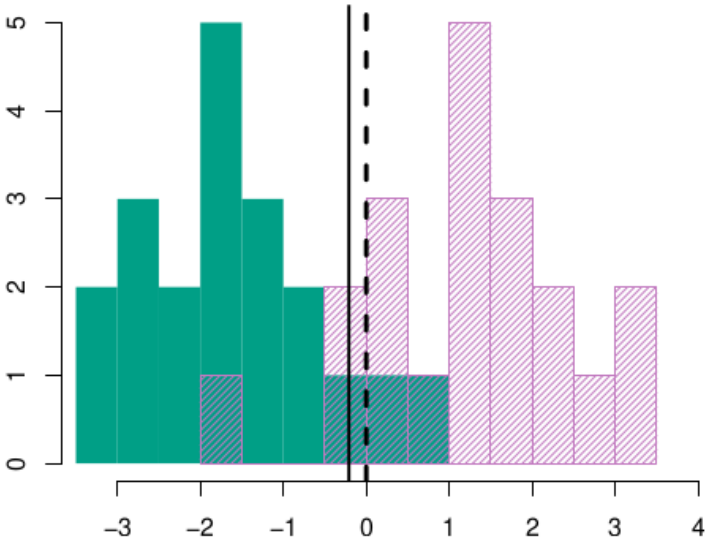
Inference phase

Given the input x, **discriminant function** is computed for all $c \in \{0, 1, 2, ..., C\}$:

$$g_c(x) = x^T \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^T \hat{\Sigma}^{-1} \hat{\mu}_c + \log p(y=c)$$

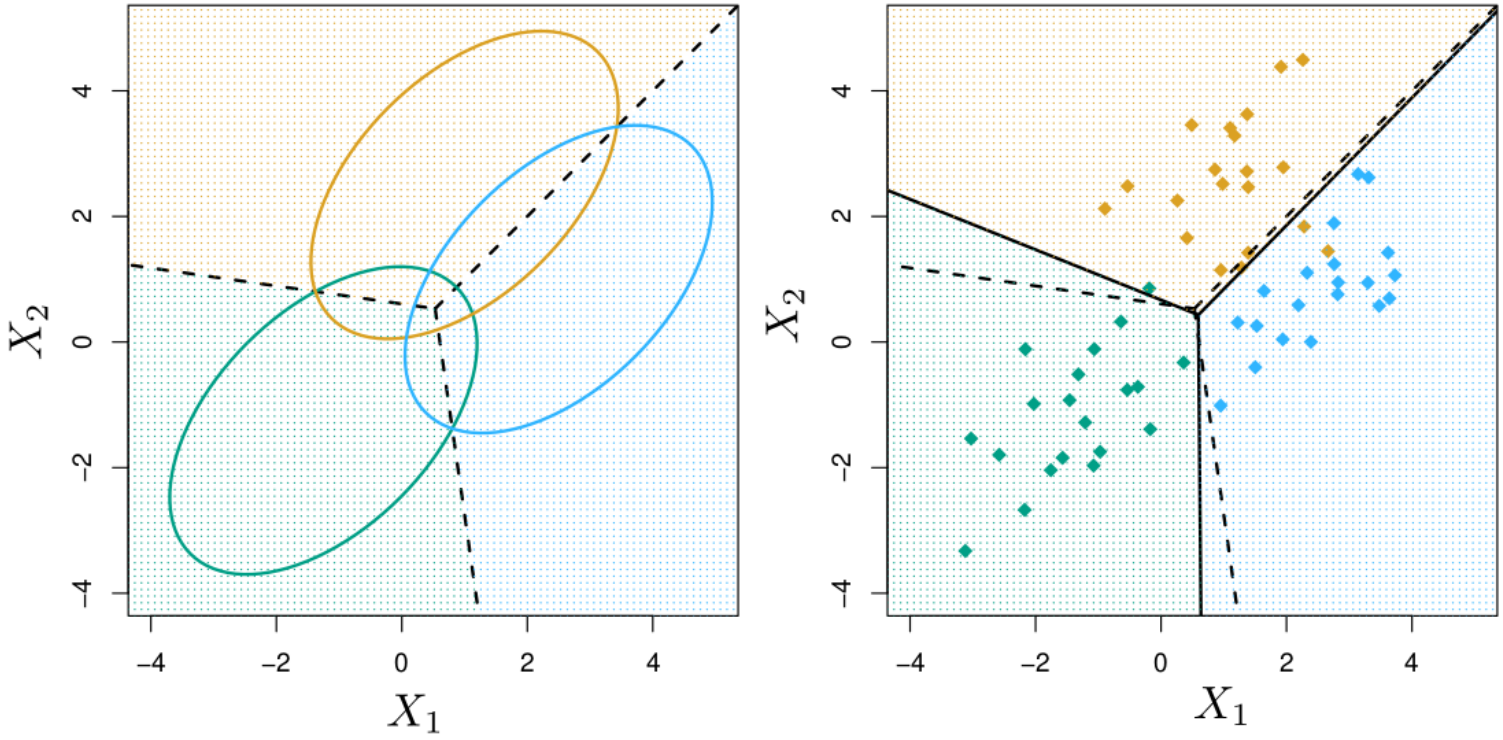
LDA decision boundaries for scalar x





- **Setting:** Let the true densities be Gaussian distributions: $P(x|y=0)$ with $\mu_1 = -1.25$ and $P(x|y=1)$ with $\mu_2 = +1.25$ and unit variances for both. Draw 20 observations from each class.
- One can use Bayes decision (left) is then $x = (\mu_1 + \mu_2)/2 = 0$, which is unknown in real-life.
- The area for which two densities overlapped is the classification error
- LDA decision boundary (solid line) is a little off from Bayes since it uses the sample mean computed from the training data.

LDA decision boundaries for a vector of x



- **Setting:** 3-class, true distribution is Gaussian with equal Σ, 20 samples for each class
- Bayes decision boundary and 95% confidence ellipsoid (use true μ_c, Σ).
- Solid lines are LDA decision boundaries $(\hat{\mu}_c, \hat{\Sigma})$. Dashed lines are Bayes.

Quadratic discriminant analysis (QDA)

Assumption.
QDA assumes observations in each class are Gaussian but each has its own covariance matrices — the posterior probability $p_c(x) \triangleq P(y = c|x)$ is

$$p_c(x) = \left[\frac{\exp\left[-\frac{1}{2}(x-\mu_c)^T \Sigma_c^{-1}(x-\mu_c)\right] \cdot \pi_c}{(2\pi)^{n/2}(\det \Sigma_c)^{1/2}} \right] / p(x)$$

Inference.
QDA follows Bayes classifier to perform the prediction, choosing the class l where

$$\text{Discriminant func.: } g_c(x) = -\frac{1}{2}(x-\mu_c)^T \Sigma_c^{-1}(x-\mu_c) - \frac{1}{2}\log\det\Sigma_c + \log\pi_c$$

is highest among $c = 1, 2, ..., C$; however, it uses estimates of μ_c, Σ_c.

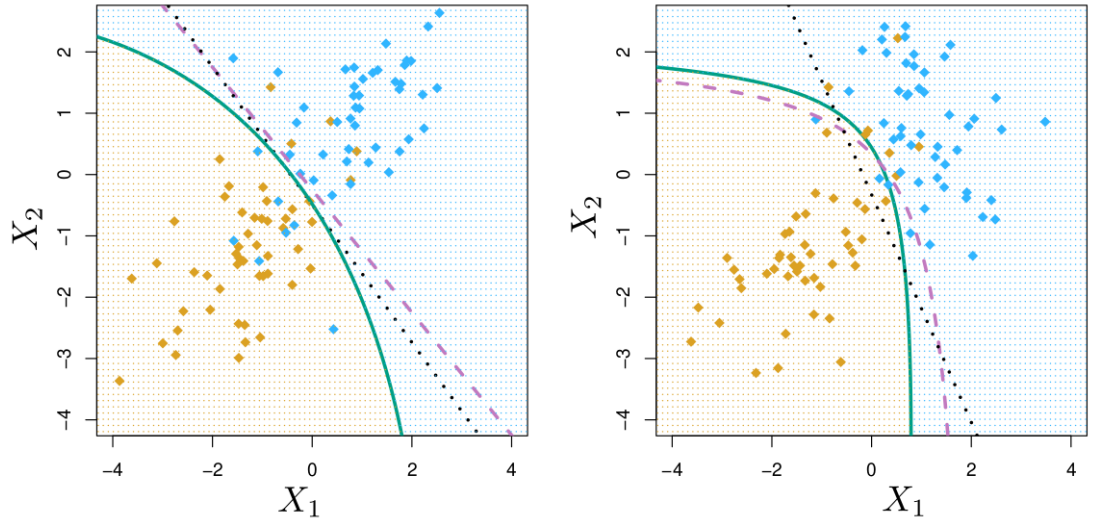
- the decision boundary between class c and k is given by the set of x that $g_c(x) = g_k(x)$ — becoming quadratic function in x.
- QDA has more number of parameters to estimate — leading the chance of getting less bias but also higher variance than LDA.

Comparison between LDA and QDA

Assume n feature dimension and C class

method	number of parameters	model property
LDA	nC	low flexibility, low variance
QDA	$n(n+1)C/2$	high flexibility, high variance

Comparison: Gaussian with *left*: common and *right*: different covariances.

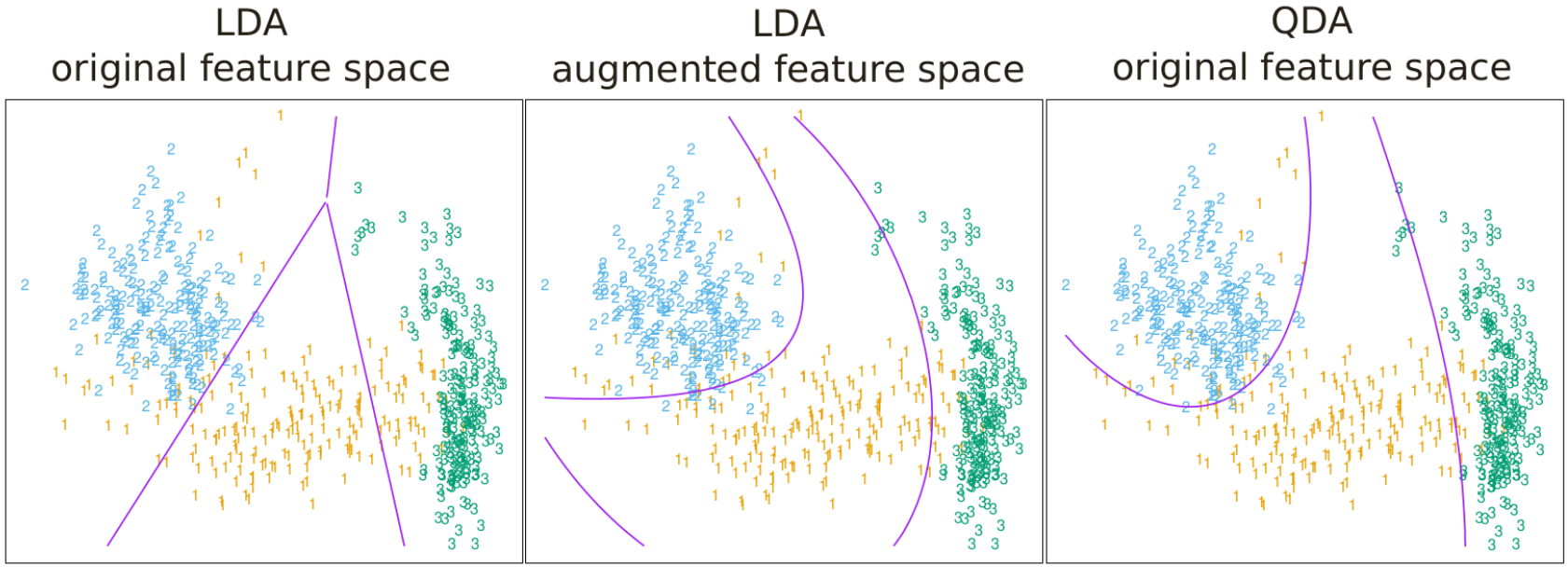


- *Left:* Bayes decision boundary is linear and accurately approx. LDA
- *Right:* now Bayes decision boundary is quadratic, and QDA is more accurate.

Augmented features in LDA vs QDA

Feature spaces: original and its mapping by a quadratic function.

Original: $\{X_1, X_2, ..., X_n\}$, Augmented: $\{X_1, X_2, ..., X_n, X_1^2, X_2^2, ..., X_n^2\}$



- *Middle*: linear functions of LDA performed on the augemented space results in quadratic functions in the original space.
- While preferring QDA, the differences between QDA and LDA (middle) are small.

Comparing LDA, QDA, and Naïve Bayes

These classifiers have a common goal, *i.e.*, to assign an observation to the class that maximizes $P(y = c|x)$.

Let's set C as a *baseline* class; equivalently, we aim at assigning an observation that maximize the followings, for $c = 1, ..., C$,

$$\log\left(\frac{P(y = c|x)}{P(y = C|x)}\right).$$

Comparing LDA, QDA, and Naïve Bayes: LDA

LDA: predictors within each class are drawn from a multivariate normal density with class-specific mean and **shared** co-variance matrix.

$$
\begin{aligned}
\log\left(\frac{P(y=c|x)}{P(y=C|x)}\right) &= \log\left(\frac{\pi_c f_c(x)}{\pi_C f_C(x)}\right) \\
&= \log\left(\frac{\pi_c \exp\left(-\frac{1}{2}(x-\mu_c)^T\Sigma^{-1}(x-\mu_c)\right)}{\pi_C \exp\left(-\frac{1}{2}(x-\mu_C)^T\Sigma^{-1}(x-\mu_C)\right)}\right) \\
&= \log\left(\frac{\pi_c}{\pi_C}\right) - \frac{1}{2}(\mu_c+\mu_C)^T\Sigma^{-1}(\mu_c-\mu_C) \\
&\quad + x^T\Sigma^{-1}(\mu_c-\mu_C) \\
&= a_c + \sum_{i=1}^{n} b_{ci}x_i
\end{aligned}
$$

where $a_c = \log\left(\frac{\pi_c}{\pi_C}\right) - \frac{1}{2}(\mu_c+\mu_C)^T\Sigma^{-1}(\mu_c-\mu_C)$ and b_{ci} is the ith component of $\Sigma^{-1}(\mu_c-\mu_C)$. Hence LDA, like logistic regression, has the log odds of the posterior probabilities is linear in x.

Comparing LDA, QDA, and Naïve Bayes: QDA

QDA has the log odds of the posterior probabilities is quadratic in x.

$$\log\left(\frac{P(y=c|x)}{P(y=C|x)}\right) = a_c + \sum_{i=1}^{n} b_{ci}x_i + \sum_{p=1}^{n}\sum_{i=1}^{n} c_{cip}x_i x_p$$

where a_c, b_{ci}, c_{cip} are functions of $\pi_c, \pi_C, \mu_c, \mu_C$ and Σ.

Comparing LDA, QDA, and Naïve Bayes: Naïve Bayes

Naïve Bayes: assume conditional independency among features given a label k.

$$\begin{aligned}
\log\left(\frac{P(y=c|x)}{P(y=C|x)}\right) &= \log\left(\frac{\pi_c f_c(x)}{\pi_C f_C(x)}\right) \\
&= \log\left(\frac{\pi_c \prod_{i=1}^n f_{ci}(x_i)}{\pi_C \prod_{i=1}^n f_{Cj}(x_i)}\right) \\
&= \log\left(\frac{\pi_c}{\pi_C}\right) + \sum_{i=1}^n \log\left(\frac{f_{ci}(x_i)}{f_{Ci}(x_i)}\right) \\
&= a_c + \sum_{i=1}^p g_{ci}(x_i)
\end{aligned}$$

where $a_c = \log\left(\frac{\pi_c}{\pi_C}\right)$ and $g_{ci}(x_i) = \log\left(\frac{f_{ci}(x_i)}{f_{Ci}(x_i)}\right)$

Softwares

Methods	Python (scikit-learn)
Logistic regression	`linear-model.LogisticRegression`
Naïve Bayes	`naive-bayes`
kNN	`neighbors.KNeighborsClassifier`
LDA, QDA	`discriminant-analysis`

References I

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://faculty.marshall.usc.edu/gareth-james/ISL/.

[3] Jitkomut Songsiri. "Statistical Learning". In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 8. Introduction to Classification. URL: http://jitkomut.eng.chula.ac.th/ee575.html.